

KonsoleH Email Verifier - Deployment Guide

This guide covers multiple deployment options for the KonsoleH Email Verifier.



Option 1: Railway (Recommended)

Railway is ideal for this application because it supports persistent volumes for SQLite.

Prerequisites

- GitHub account
- Railway account (sign up at <https://railway.app>)

Steps

1. Push to GitHub (if not already there)

```
bash
git remote add origin https://github.com/yourusername/konsoleh-verifier.git
git push -u origin main
```

2. Deploy on Railway

- Go to <https://railway.app>
- Click “New Project” → “Deploy from GitHub repo”
- Select your repository
- Railway will auto-detect the Dockerfile and deploy

3. Add Volume (Important!)

- In your Railway project, go to your service
- Click “Variables” tab
- Add: `DATABASE_URL=file:/app/data/konsoleh.db`
- Go to “Settings” tab
- Scroll to “Volumes”
- Click “Add Volume”
- Mount path: `/app/data`
- This ensures your database persists across deployments

4. Generate Domain

- Go to “Settings” tab
- Under “Networking”, click “Generate Domain”
- Your app will be live at: `yourapp.up.railway.app`

Railway CLI Deployment

Alternatively, deploy via CLI:

```
# Install Railway CLI
npm i -g @railway/cli

# Login
railway login

# Initialize project
railway init

# Deploy
railway up

# Add volume via dashboard (Settings → Volumes → Mount: /app/data)

# Generate domain
railway domain
```

Option 2: Docker (Self-Hosted)

Deploy on any server with Docker.

Build and Run

```
# Build image
docker build -t konsoleh-verifier .

# Run with persistent volume
docker run -d \
  --name konsoleh-verifier \
  -p 3000:3000 \
  -v konsoleh-data:/app/data \
  -e DATABASE_URL="file:/app/data/konsoleh.db" \
  konsoleh-verifier

# View logs
docker logs -f konsoleh-verifier
```

Docker Compose

Create `docker-compose.yml` :

```

version: '3.8'
services:
  app:
    build: .
    ports:
      - "3000:3000"
    volumes:
      - konsoleh-data:/app/data
    environment:
      - DATABASE_URL=file:/app/data/konsoleh.db
      - NODE_ENV=production
    restart: unless-stopped

volumes:
  konsoleh-data:

```

Then run:

```
docker-compose up -d
```

Option 3: Render

Render also supports persistent disks.

1. **Create account** at <https://render.com>
2. **New Web Service** → Connect your GitHub repo
3. **Build Command:** `npm install && npx prisma generate && npm run build`
4. **Start Command:** `npx prisma db push --skip-generate && npm start`
5. **Add Disk:**
 - In service settings, add a Persistent Disk
 - Mount path: `/app/data`
6. **Environment Variable:**
 - `DATABASE_URL=file:/app/data/konsoleh.db`

Option 4: VPS (DigitalOcean, Linode, etc.)

Deploy on a standard VPS with Node.js.

Prerequisites

- VPS with Node.js 18+ installed
- Nginx (optional, for reverse proxy)
- PM2 (for process management)

Steps

```
# 1. Clone repository
git clone https://github.com/yourusername/konsoleh-verifier.git
cd konsoleh-verifier

# 2. Install dependencies
npm install

# 3. Set up environment
echo 'DATABASE_URL="file:./prisma/production.db"' > .env.production

# 4. Initialize database
npx prisma generate
npx prisma db push

# 5. Build
npm run build

# 6. Install PM2
npm install -g pm2

# 7. Start with PM2
pm2 start npm --name konsoleh-verifier -- start
pm2 save
pm2 startup

# 8. Set up Nginx reverse proxy (optional)
# /etc/nginx/sites-available/konsoleh-verifier
server {
    listen 80;
    server_name your-domain.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}

# Enable site
sudo ln -s /etc/nginx/sites-available/konsoleh-verifier /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx

# 9. Set up SSL with Let's Encrypt (optional)
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d your-domain.com
```



Security Considerations

1. **HTTPS:** Always use HTTPS in production
2. **Environment Variables:** Never commit `.env` files with sensitive data
3. **Rate Limiting:** Consider adding rate limiting for the `/api/verify` endpoint

4. **Database Backups:** Regular backups of `/app/data/konsoleh.db`
 5. **SMTP Limits:** Be aware some providers may rate-limit port 25 connections
-

Monitoring

Railway

- Built-in metrics and logs in dashboard

Docker

```
docker stats konsoleh-verifier
docker logs -f konsoleh-verifier
```

PM2

```
pm2 logs konsoleh-verifier
pm2 monit
```

Updates

Railway / Render

- Push to GitHub, auto-deploys

Docker

```
git pull
docker-compose down
docker-compose up -d --build
```

VPS

```
cd konsoleh-verifier
git pull
npm install
npm run build
pm2 restart konsoleh-verifier
```

Troubleshooting

Database locked error

- Ensure only one instance is running
- Check volume permissions

SMTP connection failures

- Some hosting providers block port 25
- Consider using a different verification method or VPN

Out of memory

- Increase container memory limits
- Reduce concurrent verification workers in `src/lib/konsoleh-verifier.ts`

Support

For issues, check:

1. Application logs
2. Database connection
3. Network/firewall settings
4. Environment variables

Recommended for production: Railway or Render (easiest) or **Docker on VPS** (most control).